eBPF内核栈解析特性现状及发展

The 4th eBPF Developer Conference (eBPFDC 2026)

陈涛

中国 · 西安　　2026年4月18日

1.栈解析功能引入的背景及必要性

2.eBPF子系统中栈相关基础API

3.栈解析功能的问题与挑战

4.社区未来发展趋势

1.栈解析功能引入的背景及必要性

1.1为什么“抓栈”很重要

在系统诊断、性能分析和故障定位中，知道某个事件发生了通常还不够，调用栈正是连接“现象”和“根因”的核心信息：

1.CPU 为什么高？要看热点函数是被谁调用起来的
2.某个 I/O 请求为什么慢？要看它卡在了哪一层调用链路上
3.锁竞争、内存分配、网络延迟为什么异常？往往也需要借助调用栈做归因

事件只能告诉我们发生了什么，栈才能进一步告诉我们为什么发生、是谁触发的

1.2 传统抓栈手段为什么不够

在 eBPF 普及之前，抓栈的常见方式包括：

1.perf 这类采样分析工具
2.gdb、pstack 等调试工具
3.在业务代码里手动埋点
4.内核模块或专用探针工具

传统方法的一些限制：

1.侵入性强：需要改代码、加日志、重新部署。
2.覆盖面有限：很多时候只能观察某一层，难以同时贯通内核态和用户态。
3.动态性不足：线上临时排障时，不可能总是提前加好所有埋点。
4.成本较高：某些调试方式对性能影响较大，不适合长期在线运行。

1.3 eBPF为什么适合抓栈

允许开发者把一小段经过校验的程序挂到内核中的关键事件点上，比如：

1.profile 定时采样点
2.kprobe / tracepoint
3.uprobes

eBPF 可以在事件发生时抓取内核栈和用户栈，再把栈信息和 CPU、进程、线程、时间、事件类型等上下文关联起来

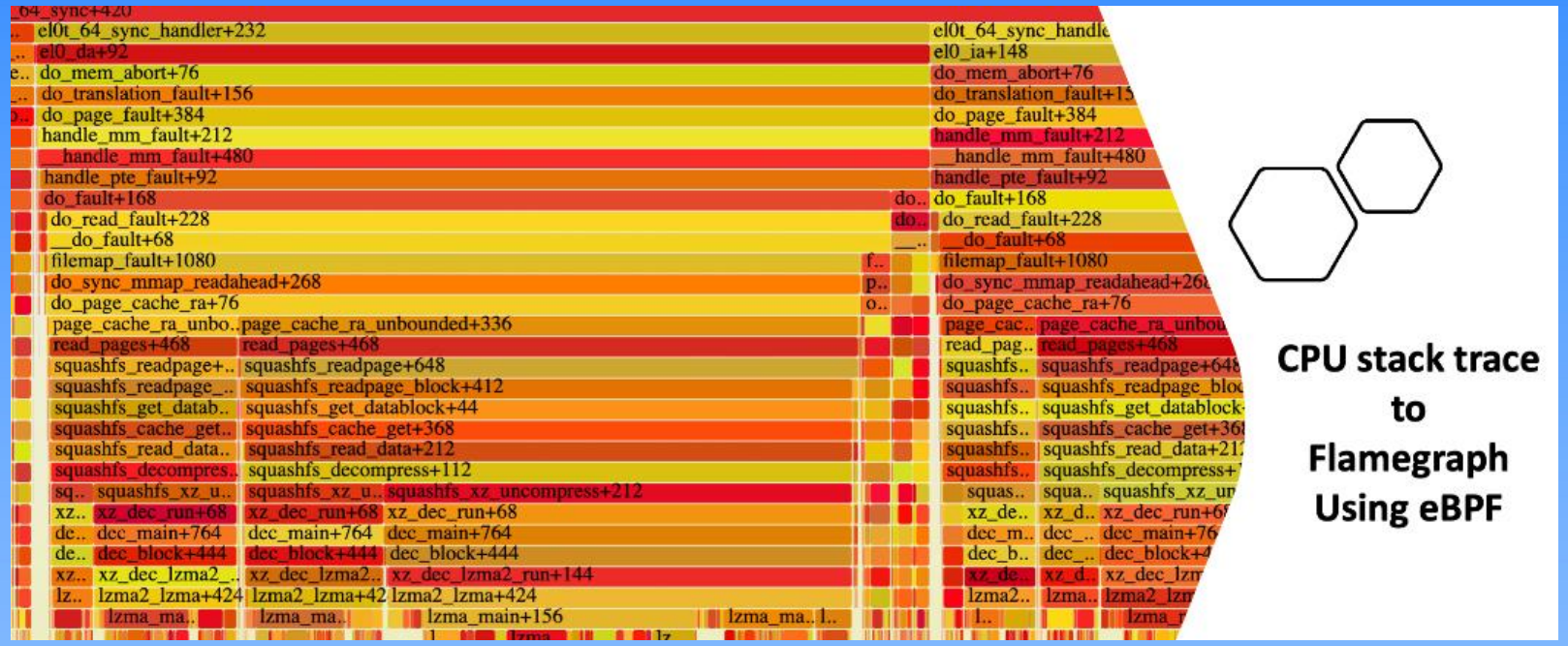
eBPF 抓栈能力是现代可观测性体系从“指标观测”走向“路径归因”的关键基础能力之一

1.4 eBPF抓栈功能在开源项目中的相关应用

1.BCC （profile、offcputime、stackcount、funclatency）
2.bpftrace

3.pyroscope
4.parca agent

5.pixie
6.datadog
7.deepflow



https://israelo.io/blog/getting-started-with-ebpf-tracing/

2.eBPF子系统中栈相关基础API

2.1 bpf_get_stackid

[ip1，ip2，ip3，ip4]

计算hash值

在stack_trace_map中查找

找到相同的栈 -> 返回旧的stack_id

没找到 -> 新建entry，分配新的stack_id

A. 容易发生哈希冲突与MAP 容量限制

B. 需要二次查询才能获取真正的栈

BPF_F_FAST_STACK_CMP

BPF_F_REUSE_STACKID

2.2 bpf_get_stack

[ip1, ip2, ip3, ip4]

通过Ring Buffer等直接output给用户

A.数据拷贝开销

与 bpf_get_stackid 只返回一个 4 字节的 ID 不同，bpf_get_stack 会拷贝整个地址数组

B.无内置去重

bpf_get_stack 存在重复搬运相同栈的情况

2.3 其他

bpf_get_task_stack

```
SEC( “iter/task” )
{
    bpf_get_task_stack();
}
```

bpf_get_stack_sleepable

```
SEC( “lsm.s/socket_bind” )
{
    bpf_get_stack();
}
```

```
return prog->sleepable ?
&bpf_get_stack_sleepable_proto :
&bpf_get_stack_proto;
```

bpf_get_stack{id_pe, _pe}

```
SEC(“perf_event”)
{
    bpf_get_stack();
}
```

```
pe_prog_func_proto(enum bpf_func_bpf_prog
                   *prog)
{
    switch (func_id) {
    case BPF_FUNC_get_stackid:
        return &bpf_get_stackid_proto_pe;
    case BPF_FUNC_get_stack:
        return &bpf_get_stack_proto_pe;
```

3.栈解析功能的问题与挑战

3.1 stack_trace_map 的不可靠性

用户态获取栈id获取栈信息

```
bpf_map_lookup_elem(stack_id, value);
bpf_map_delete_elem(stack_id);
```

```
bucket = xchg(&smap->buckets[id], NULL);
memcpy(value, bucket->data, trace_len);
old_bucket = xchg(&smap->buckets[id], bucket);
if (old_bucket)
    pcpu_freelist_push(&smap->freelist, &old_bucket->fnode);
```

存在新值写入old_bucket但又被bucket覆盖

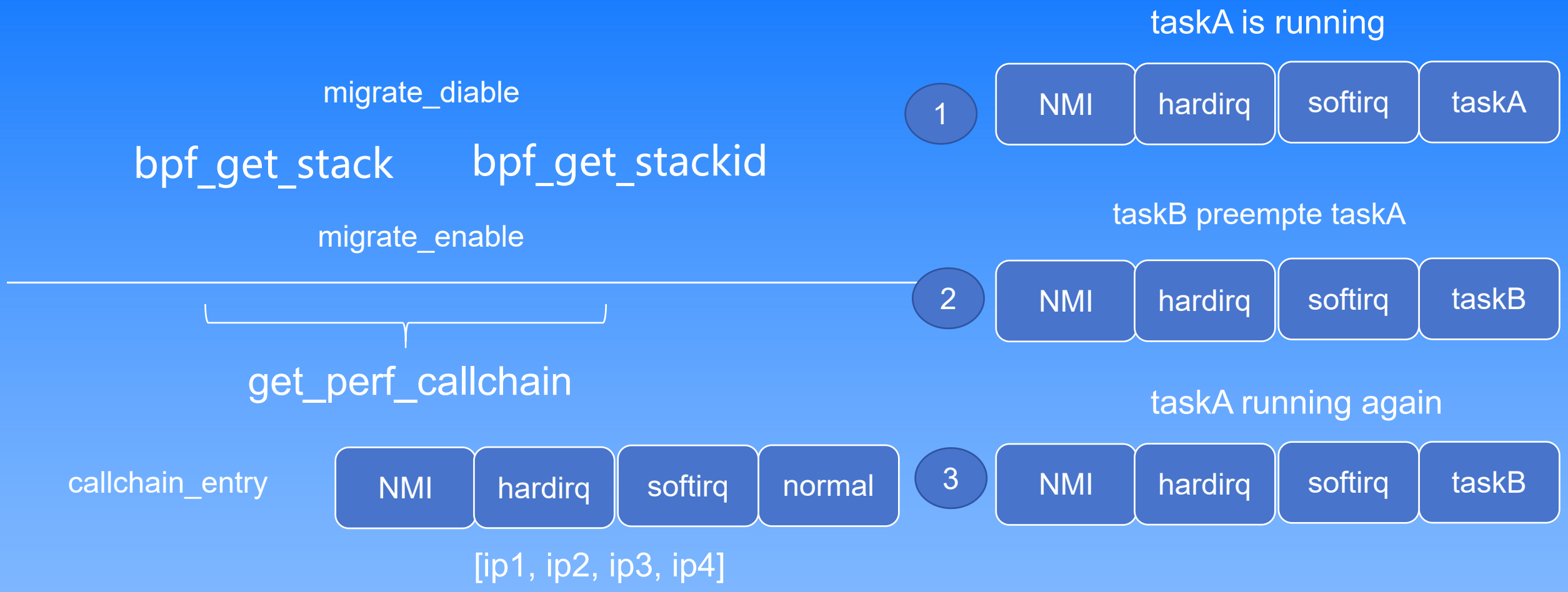
合并接口，获取后立即删除

```
bpf_map_lookup_and_delete_elem(stack_id, value);
```

```
bucket = xchg(&smap->buckets[id], NULL);
memcpy(value, bucket->data, trace_len);
```

https://lore.kernel.org/bpf/CAEf4BzZ2Fg+AmFA-K3YODE27br+e0-rLJwn0M5XEwfEHqpPKgQ@mail.gmail.com/

3.2 抢占情况下栈可能不准确



3.3 用户栈展开可靠性不足

eBPF原生采栈方案依赖 FP 链（%rbp 寄存器链）进行栈展开（stack unwinding），而现代编译器（如 GCC/Clang 的 -O2/-O3 优化）默认会启用 -fomit-frame-pointer 优化，将 %rbp 用作通用寄存器，导致栈回溯中断或失败。

```
int a(void)
{
        return b() - 1;
}

int b(void)
{
        return c() + 1;
}

int c()
{
        memset(buf, '+', sizeof(buf)-1);

        while (1) {
                size_t len = getlen(buf);
                assert(len == sizeof(buf) - 1)
        }

        // not reached
        return 1;
}

int main(int argc, char **argv)
{
        a();
}
```

```
#0  0x0000007fbe74cc20 strerrordesc_np+0x1980 (/u
#1  0x0000007fbe6db30c __libc_start_main+0x9c (/u
#2  0x000000556f9c0754 _start+0x34 (/var/rootdirs
-                test-strlen-abc (83030)
    290
```

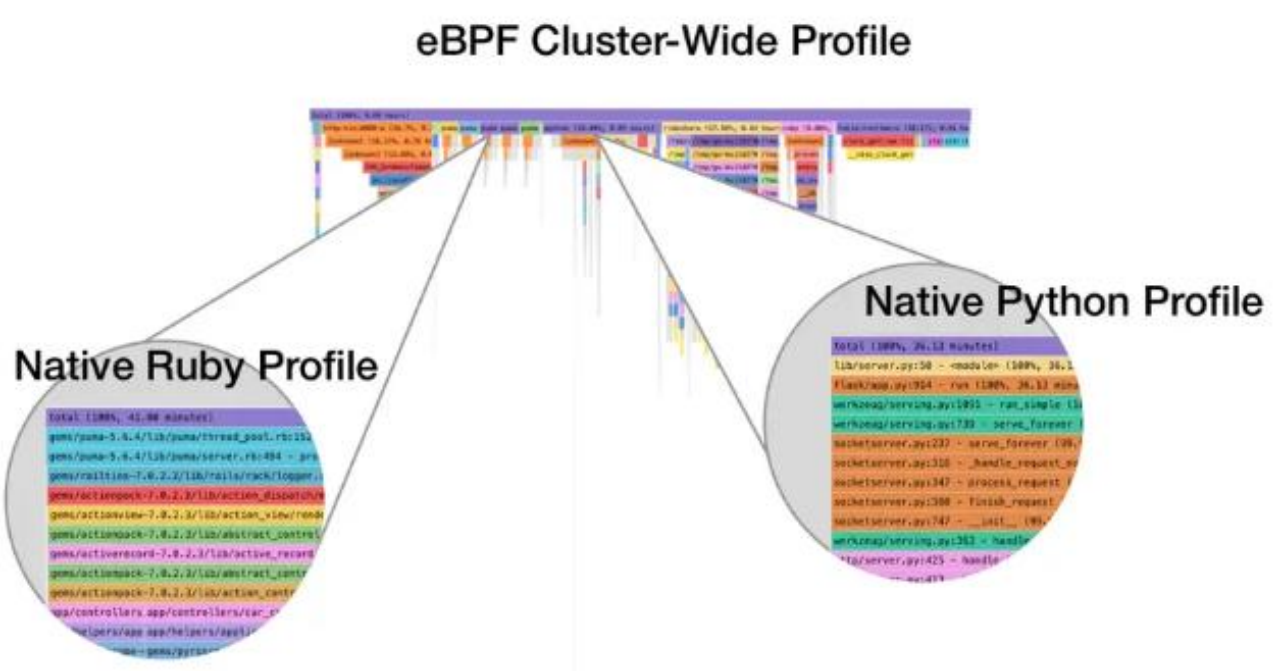
```
#0  0x0000007fbe74cc20 strerrordesc_np+0x1980 (/u
#1  0x000000556f9c087c c+0x30 (/var/rootdirs/home
#2  0x000000556f9c0840 b+0x8 (/var/rootdirs/home/
#3  0x000000556f9c082c a+0x8 (/var/rootdirs/home/
#4  0x000000556f9c08c4 main+0x10 (/var/rootdirs/h
#6  0x0000007fbe6db230 __libc_init_first+0x70 (/u
-                test-strlen-abc (83030)
```

use unwind table (.eh_frame)

https://github.com/iovisor/bcc/pull/4463

3.4 多语言运行时支持不一致

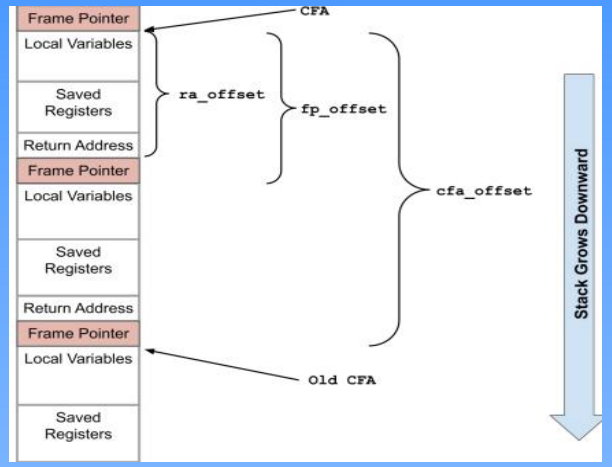
C/C++ 相对直接
Go、Java、Python、Rust async 场景更复杂



4.社区未来发展趋势

4.1 SFrame & BPF

SFrame 的设计初衷是提供一种快速、低开销、可靠的栈跟踪机制，作为帧指针（Frame Pointer）和 DWARF .eh_frame 的替代方案 。它由 Oracle 内部团队发起，后成为 GNU 工具链和 Linux 内核的正式组件。




```
From: Steven Rostedt <rostedt@goodmis.org>
To: bpf@vger.kernel.org, Alexei Starovoitov <ast@kernel.org>,
        Andrii Nakryiko <andrii@kernel.org>,
        Ian Rogers <irogers@google.com>,
        Blake Jones <blakejones@google.com>
Subject: BPF with deferred stacktrace and sframes?
Date: Mon, 26 Jan 2026 14:21:18 -0500   [thread overview]
Message-ID: <20260126142118.2ea3cf13@gandalf.local.home> (raw)

Hi folks,

Now that deferred stacktrace is upstream and hopefully sframes will soon
follow, is there any plans to integrate BPF with this facility?

Would there be a way to register a BPF program to request and receive a
user space backtrace when the task goes back to user space?

Basically the API would require a program to register a callback function
that retrieves the backtrace at a later time, and then during runtime (in
any context) request that function to be called when the currently running
task goes back to user space.

I've been Cc'ing the BPF list (and maintainers) on the deferred stack trace
work and did get a few comments. But I haven't heard anything about BPF
utilizing the feature yet. So I'm asking now if that's in the works?
```

bpf_get_stack is synchronous
SFrame is asynchronous

https://lore.kernel.org/lkml/cover.1730150953.git.jpoimboe@kernel.org/
https://lore.kernel.org/bpf/20260126142118.2ea3cf13@gandalf.local.home/

Thanks